

Q ARCHITECTURE DEEP DIVE

SEMANTIC SEARCH

VS

KEYWORD SEARCH

VS

HYBRID

Why neither alone survives production
and what actually works



AI Coach John



FOLLOW

↻ Repost

The Search Problem Nobody Talks About

Your search works perfectly in demos. It breaks in production. Here's why.



REAL SCENARIO

You build a product search for an e-commerce platform. You pick semantic search because it sounds modern and powerful.



WEEK 1 IN PRODUCTION

- User searches "**Nike Air Max 270**" (exact product name)
- Semantic search returns "Adidas Ultraboost" and "running shoes comfort"
- The exact product exists in your catalog. The user can't find it.
- Conversion drops. Users blame search. You blame embeddings.



THE REAL PROBLEM

- Semantic search finds **meaning**. Keyword search finds **exact matches**.
- Production users need both sometimes in the same query.
- Neither alone is enough. This is why **Hybrid** exists.



How Keyword Search Works

Keyword search = find documents that contain the exact words you typed

⚙️ THE ENGINE: BM25

- Ranks documents based on term frequency and inverse document frequency
- Words that appear often in a document but rarely in the corpus score higher
- No understanding of meaning purely statistical matching

✅ WHAT IT'S GOOD AT

- Exact product names, model numbers, SKUs
- Technical terms, error codes, proper nouns
- Legal/medical terminology that must match precisely

📊 VISUAL: BM25 SCORING

Query: "python async programming"

Doc A: "Python async with asyncio"
→ Score: 8.4 ✓ HIGH

Doc B: "Async patterns in software"
→ Score: 3.1 ✗ LOW

Doc C: "Python web dev guide"
→ Score: 4.2 ✗ LOW

✗ WHAT IT MISSES

- **Synonyms** "buy" vs "purchase"
- **Intent** "for headache" should return paracetamol
- **Multilingual** query in Hindi, document in English



How Semantic Search Works

Find documents that mean what you're looking for, even if they use different words

THE ENGINE: DENSE VECTORS

- Both query and documents are converted into high-dimensional vectors
- Vectors closer together in space = more similar in meaning
- Retrieved by cosine similarity or approximate nearest neighbor

WHAT IT'S GOOD AT

- **Intent understanding** "help me sleep" matches melatonin
- **Synonym handling** "cheap" = "affordable"
- **Multilingual** matches meaning across languages

VISUAL: VECTOR SPACE

"buy a car" — "purchase auto"
(very close) sim: 0.94

"buy a car" "engine repair"
(far apart) sim: 0.21

WHAT IT MISSES

- **Exact matching** "GPT-4o" vs "GPT-4"
- **Rare terms** domain-specific terms
- **Numeric precision** "rate 7.5%" vs "8.2%"



Head to Head: Where Each Breaks

Failure Mode Comparison Chart

QUERY TYPE	KEYWORD	SEMANTIC	WINNER
"Nike Air Max 270"	✓ HIGH	⚠ MED	Keyword
"shoes for bad knees"	✗ LOW	✓ HIGH	Semantic
"error code 404 fix"	✓ HIGH	⚠ MED	Keyword
"something to calm anxiety"	✗ LOW	✓ HIGH	Semantic
"iPhone 15 Pro Max"	✓ HIGH	⚠ MED	Keyword
"gift for my girlfriend"	✗ LOW	✓ HIGH	Semantic
"7.5% fixed rate mortgage"	✓ HIGH	✗ LOW	Keyword

THE PATTERN

- Exact terms, numbers, product names, codes Keyword wins
- Intent, meaning, natural language, synonyms Semantic wins
- Real users search both ways in the same session. **Pick one and you fail half your users.**



AI Coach John



FOLLOW

 Repost

How Hybrid Search Works

Run both BM25 and semantic search, fuse the results into one ranked list

THE PIPELINE

User Query

↓ Parallel Retrieval

BM25 Search

Doc A: 8.4
Doc C: 6.1
Doc B: 3.2

Vector Search

Doc C: 0.94
Doc A: 0.87
Doc D: 0.71



Score Fusion Layer
Reciprocal Rank Fusion (RRF)



Final Ranked Results:

1. Doc A (high in both)
2. Doc C (high in both)
3. Doc D (semantic only)
4. Doc B (keyword only)

THE FUSION ALGORITHM (RRF)

→ Reciprocal Rank Fusion

→
$$\text{Score} = 1/(\text{rank_in_bm25} + k) + 1/(\text{rank_in_semantic} + k)$$

→ Documents that rank high in BOTH systems get a massive combined boost

→ Documents strong in only one system still surface but rank lower

→ $k = 60$ is the standard constant



RRF Scoring Visualized

How Reciprocal Rank Fusion combines rankings

DOCUMENT	BM25 RANK	SEMANTIC RANK	RRF SCORE
Doc A	#1	#2	0.031
Doc C	#2	#1	0.031
Doc D	#8	#3	0.024
Doc B	#3	#12	0.021
Doc E	#15	#4	0.019

Formula: $1/(\text{rank} + 60) + 1/(\text{rank} + 60)$

Doc A = $1/(1+60) + 1/(2+60) = 0.0164 + 0.0161 = 0.031$

👍 WHAT RRF DOES WELL

- No need to normalize BM25 and cosine similarity scores to the same scale
- Robust to outliers one bad rank in one system doesn't tank the whole result
- Simple to implement, consistently outperforms weighted score blending



AI Coach John



FOLLOW

↻ Repost

Production Performance: The Numbers

Retrieval Accuracy by Query Type (NDCG@10)

QUERY TYPE	BM25	SEMANTIC	HYBRID
Exact term queries	0.82	0.61	0.85 ▲
Intent queries	0.34	0.79	0.83 ▲
Mixed queries	0.51	0.63	0.81 ▲
Rare term queries	0.71	0.44	0.74 ▲
Long queries	0.42	0.74	0.79 ▲
Overall	0.56	0.64	0.80 ▲

💡 KEY TAKEAWAY

- Hybrid doesn't just average the two it outperforms both on almost every query type.
- On mixed queries (the majority of real production traffic): 0.81 vs 0.63 semantic vs 0.51 BM25.
- The only time hybrid doesn't win cleanly is pure exact-term queries and even then it's comparable.



AI Coach John



FOLLOW

↻ Repost

Implementing Hybrid Search

Implementation with Qdrant (production-grade vector DB)

```
from qdrant_client import QdrantClient
client = QdrantClient("localhost", port=6333)

# 1. Index docs (dense + sparse)
client.upsert(
    collection_name="products",
    points=[{
        "id": doc_id, "payload": {"text": doc},
        "vector": {"dense": dense_vec, "sparse": sparse_vec}
    }]
)

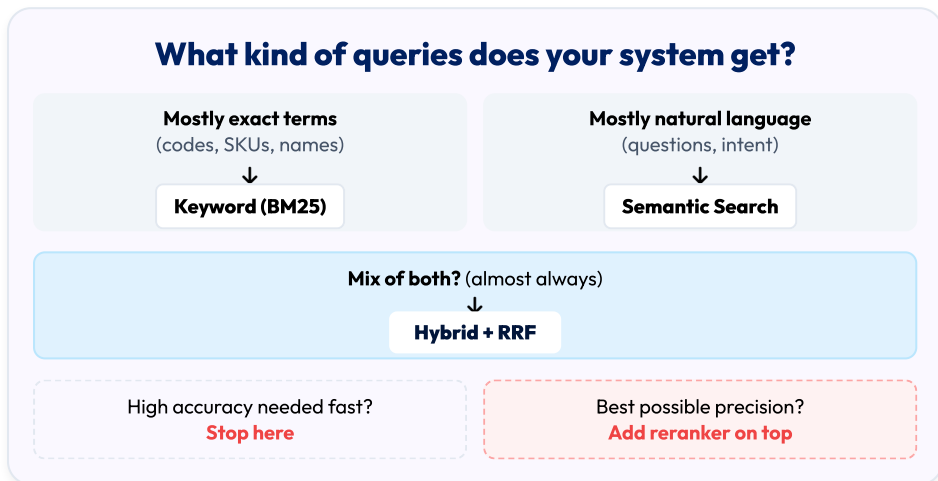
# 2. Query both & fuse with RRF
results = client.query_points(
    collection_name="products",
    prefetch=[
        {"query": d_query, "using": "dense", "limit": 20},
        {"query": s_query, "using": "sparse", "limit": 20},
    ],
    query={"fusion": "rrf"}, limit=5
)
```

ALTERNATIVE IMPLEMENTATIONS

- **Elasticsearch / OpenSearch** built-in hybrid search with RRF
- **Weaviate** BM25 + vector search native hybrid
- **LangChain EnsembleRetriever** combines any two retrievers with configurable weights

When to Use Each in Production

The Decision Framework



✓ PRODUCTION RULES

- Internal enterprise **Hybrid**
- RAG for support bots **Hybrid**
- Simple FAQ exact **BM25**
- E-commerce search **Hybrid**
- Creative / exploratory **Semantic**



AI Coach John



FOLLOW

↻ Repost

Adding a Reranker on Top

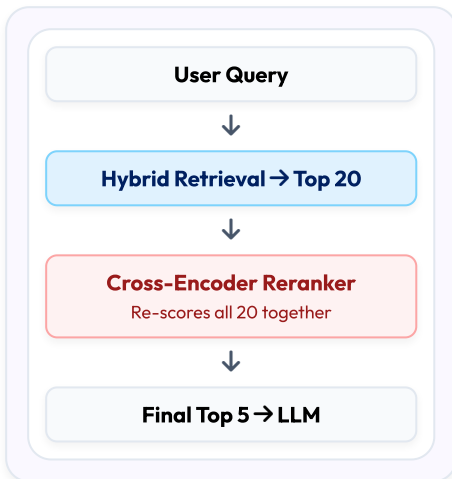
Hybrid gives you candidates. A reranker picks the best one.

⚙️ WHY RERANKING MATTERS

- BM25 and semantic search use **bi-encoders** (query and document embedded separately)
- A **cross-encoder reranker** reads query + document together much more accurate
- Run it only on your top 20 hybrid results reorder into final top 5

⌚ LATENCY TRADEOFF

- Hybrid alone: **~50ms**
- Hybrid + reranker: **~200ms**
- For RAG and high-stakes retrieval: always worth it



```

from sentence_transformers import
CrossEncoder

reranker = CrossEncoder("ms-marco-MiniLM-
L-6-v2")
pairs = [[query, doc] for doc in
hybrid_top_20]
scores = reranker.predict(pairs)

# Sort by new score
reranked = sorted(zip(hybrid_top_20,
scores), key=lambda x: x[1],
reverse=True)
final_top_5 = [doc for doc, score in
reranked[:5]]
  
```



Cheatsheet

Full Comparison Table

CAPABILITY	KEYWORD	SEMANTIC	HYBRID
Exact match	✓ Best	✗ Okay	✓ Best
Intent/meaning	✗ Poor	✓ Best	✓ Best
Synonyms	✗ Poor	✓ Best	✓ Best
Numbers/codes	✓ Best	✗ Poor	✓ Best
Cost	✓ Lowest	✗ Medium	✗ Higher
Verdict	Baseline	Good start	Production

☰ THE PRODUCTION STACK

Hybrid retrieval (BM25 + Dense + RRF)

+

Cross-encoder reranker on top 20

↓

Feed final top 5 into LLM

This is what Perplexity, Notion AI, and enterprise RAG systems run in production.



AI Coach John



FOLLOW

↻ Repost